SABARAGAMUWA UNIVERSITY OF SRI LANKA

Faculty of Computing | Department of Software Engineering

BSc Honours Degree Programme in Software Engineering

Implementation Progress Report

SE 8101 -- Research Project

Student Name:	S.Shanthosh
Index Number:	20APSE4882
Supervisor:	Mrs. WMLS Abeythunga
Date:	27th May 2026
Report Type:	Daily Implementation Update

RESEARCH TOPIC

An Empirical Evaluation of Deep Learning Models for Enhanced and Scalable Bug Detection and Classification in Large-Scale Software Systems

1. Overview of Today's Progress

This report documents the implementation work completed on 27th May 2026, covering four major phases: environment configuration, exploratory data analysis, data preprocessing, and model training. A total of six models were trained and evaluated -- four traditional machine learning models and two deep learning variants.

Phase	Task	Status	Output
1	Environment setup & library installation	Done	All libraries ready
2	**Dataset acquisition (JM1 -- PROMISE)**	**Done**	**10,885 samples**
3	Exploratory Data Analysis (EDA)	Done	4 charts saved
4	**Data Preprocessing pipeline**	**Done**	**6 split files saved**
5	Baseline ML models (RF + Naive Bayes)	Done	Best F1: 0.4265
6	**CNN-LSTM Deep Learning model**	**Done**	**Best Recall: 0.755**

2. Dataset Description

2.1 JM1 Dataset (PROMISE Repository)

The JM1 dataset, sourced from the PROMISE software engineering repository, was selected as the primary dataset. It is a NASA software defect dataset containing software modules written in C, with 21 complexity metrics as features and a binary defect label.

Property	Value
Total software modules	10,885
Number of features	**21 software complexity metrics**
Defective modules	2,106 (19.3%)
Non-defective modules	**8,779 (80.7%)**
Class imbalance ratio	4.2 : 1
Missing values	**25 (in 5 columns)**
Feature types	All numeric

2.2 Feature Description

The 21 features comprise Halstead complexity metrics and McCabe's cyclomatic complexity measures:

* loc -- Lines of Code
* v(g) -- Cyclomatic Complexity (branching paths)
* ev(g), iv(g) -- Essential and Design Complexity
* n, v, l, d, i, e, b, t -- Full Halstead metrics suite
* lOCode, lOComment, lOBlank -- Source line category counts
* uniq_Op, uniq_Opnd, total_Op, total_Opnd -- Operator/operand counts
* branchCount -- Number of branching points (if/else/switch)

3. Exploratory Data Analysis

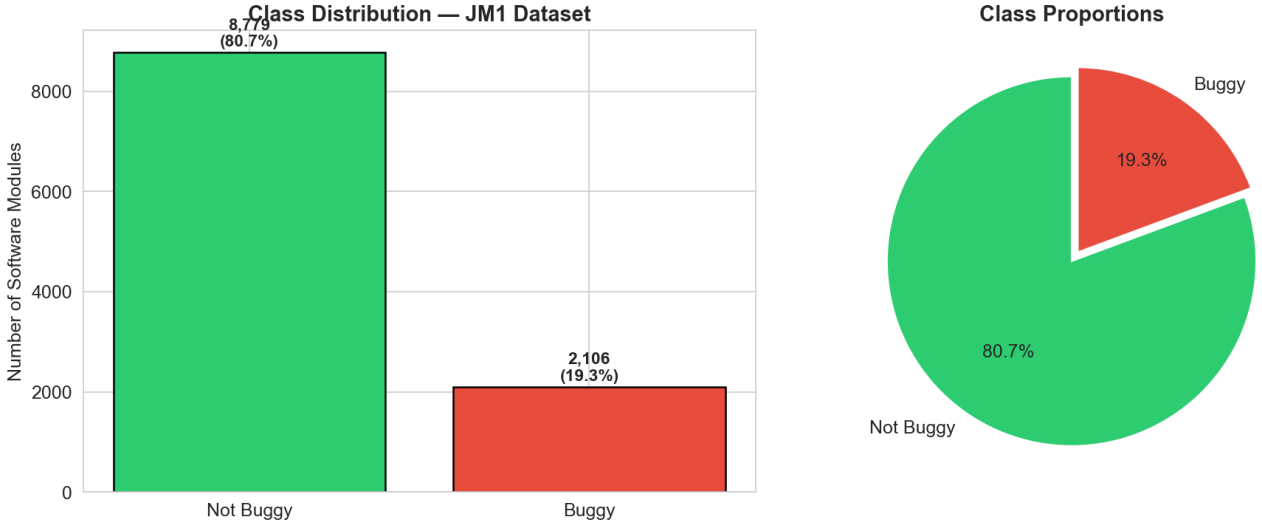
3.1 Class Imbalance

EDA confirmed a significant class imbalance: defective modules represent only 19.3% of all samples. A naive classifier predicting all modules as "non-defective" would still achieve 80.7% accuracy while catching zero bugs. This confirms that F1-Score and Recall must be used as primary evaluation metrics.

3.2 Feature Analysis -- Top Predictors

Feature	Buggy Mean	Clean Mean	Ratio	Insight
t (Halstead Time)	6,285.2	1,029.6	6.1x	Very high in buggy
e (Halstead Effort)	**113,134**	**18,533**	**6.1x**	**Very high in buggy**
v (Halstead Volume)	1,422.4	494.2	2.9x	Very high in buggy
loc (Lines of Code)	**80.4**	**32.8**	**2.4x**	**Very high in buggy**
branchCount	21.6	8.8	2.4x	Very high in buggy
v(g) (Cyclomatic)	**11.9**	**5.0**	**2.4x**	**Very high in buggy**

Figure 1: Class distribution (bar chart and pie chart)



4. Data Preprocessing Pipeline

1. Missing Value Imputation

25 missing values across 5 columns filled using column-wise median imputation, which is robust against skewed distributions common in software metrics.

2. Outlier Handling

Winsorization applied at the 99th percentile per feature to cap extreme values without removing data rows.

3. Stratified Train / Test Split

80% training (8,708 samples) and 20% test (2,177 samples), stratified by class label (random_state=42) to preserve the original imbalance ratio in both sets.

4. Min-Max Normalization

All features scaled to [0, 1]. Scaler fitted only on training set to prevent data leakage into the test set.

5. BorderlineSMOTE

Applied only to the training set. Generated 5,338 synthetic minority-class samples, achieving a balanced 1:1 ratio (7,023 buggy : 7,023 clean). Test set intentionally left at original imbalanced ratio.

Split	Total Samples	Buggy	Clean
Train (raw)	8,708	1,685	7,023
Train (SMOTE)	**14,046**	**7,023**	**7,023**
Test (real)	2,177	421	1,756

5. Baseline Machine Learning Models

5.1 Models Implemented

Two traditional ML classifiers were trained as baselines, each under two conditions: without SMOTE (imbalanced training data) and with SMOTE (balanced training data) -- giving four baseline experiments in total.

* Random Forest: 200 estimators, max depth 15, random_state=42
* Naive Bayes: Gaussian NB with default parameters

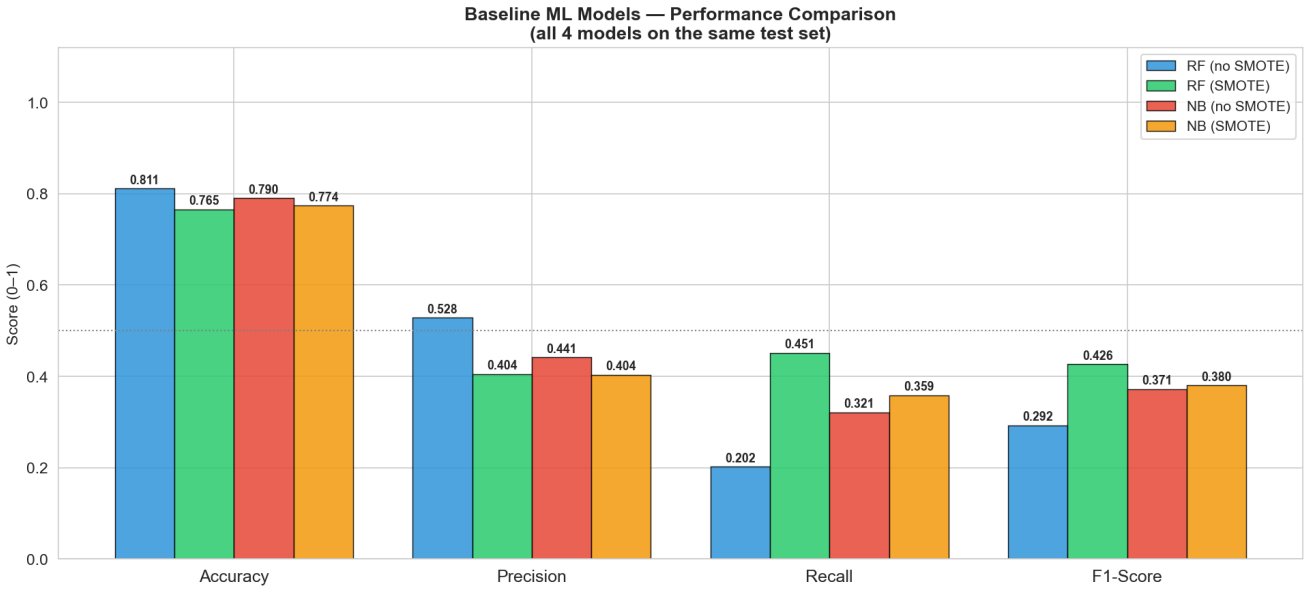
5.2 Baseline Results

Model	Accuracy	Precision	Recall	F1-Score	AUC
RF (no SMOTE)	0.8107	0.5280	0.2019	0.2921	0.7365
RF (SMOTE)	**0.7653**	**0.4043**	**0.4513**	**0.4265**	**0.7350**
NB (no SMOTE)	0.7901	0.4412	0.3207	0.3714	0.6800
NB (SMOTE)	0.7735	0.4037	0.3587	0.3799	0.6792

5.3 Key Observations

* RF + SMOTE achieved the highest F1-Score of 0.4265 -- the baseline target to beat.
* Without SMOTE, RF achieved 81.1% accuracy but caught only 20.2% of bugs (85/421), demonstrating the accuracy paradox in imbalanced classification.
* SMOTE improved RF Recall from 0.202 to 0.451 (+123%), confirming its effectiveness.
* Random Forest significantly outperformed Naive Bayes across all metrics.

Figure 2: Baseline model performance comparison



6. CNN-LSTM Deep Learning Model

6.1 Architecture

Layer	Configuration	Purpose
Input	(batch, 21, 1)	21 metric features per sample
Conv1D	**64 filters, kernel=3**	**Local pattern detection**
BatchNorm	--	Training stabilisation
Conv1D	**128 filters, kernel=3**	**Deep feature extraction**
BatchNorm	--	Training stabilisation
LSTM	**hidden=64, layers=1**	**Sequential dependencies**
Dropout	rate=0.4	Regularisation
Dense	**32 units, ReLU**	**Feature compression**
Dense	1 unit, Sigmoid	Bug probability [0, 1]
Parameters	**77,121 trainable**	**--**

6.2 Focal Loss -- Research Contribution

Two loss functions were evaluated. Binary Cross-Entropy (BCE) served as the standard deep learning baseline. Focal Loss (Lin et al., 2017) was adapted as the research contribution for imbalance-aware training:

FL(pt) = -a · (1 - pt)^g · log(pt)

* a = 0.75 -- weights the minority (buggy) class more heavily
* g = 2.0 -- down-weights easy examples, focuses on hard-to-detect bugs
* Both models trained for 30 epochs | Optimizer: Adam | LR: 0.001

6.3 CNN-LSTM Results

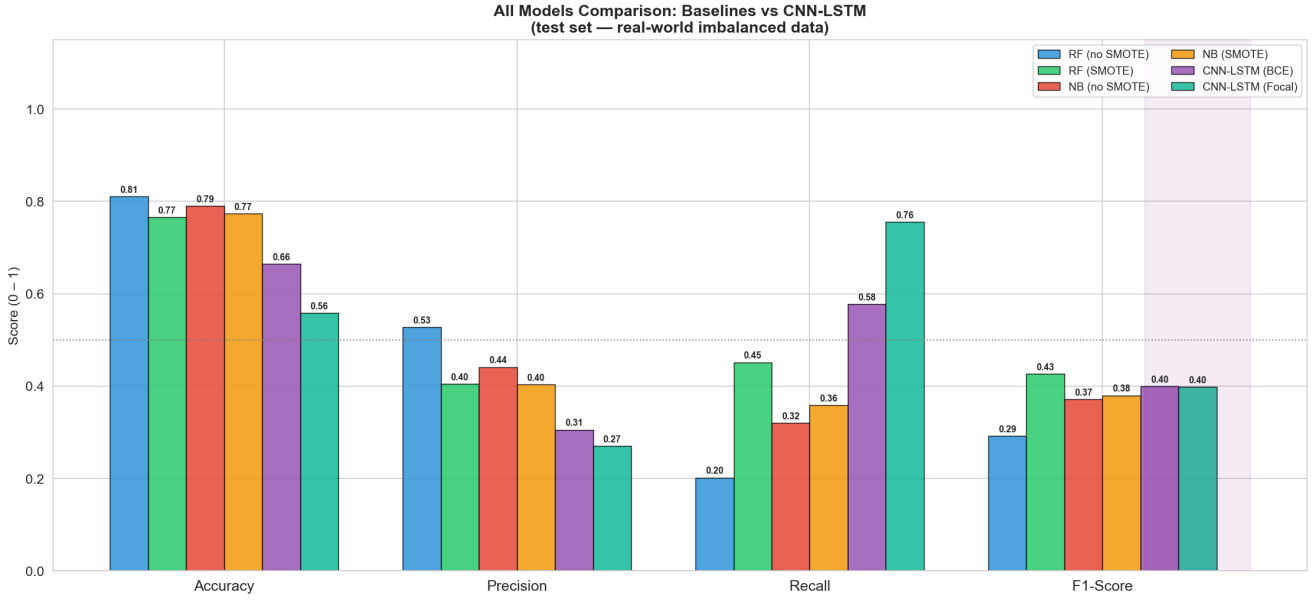
Model	Accuracy	Precision	Recall	F1-Score	AUC
CNN-LSTM (Cross-Entropy)	0.6647	0.3057	0.5772	0.3997	0.6808
CNN-LSTM (Focal Loss)	**0.5581**	**0.2702**	**0.7553**	**0.3980**	**0.7001**

6.4 Full Comparison -- All Models

Model	Accuracy	Precision	Recall	F1	AUC
RF (no SMOTE)	0.8107	0.5280	0.2019	0.2921	0.7365
RF (SMOTE)	0.7653	0.4043	0.4513	0.4265	0.7350
NB (no SMOTE)	0.7901	0.4412	0.3207	0.3714	0.6800
NB (SMOTE)	0.7735	0.4037	0.3587	0.3799	0.6792
CNN-LSTM (BCE)	0.6647	0.3057	0.5772	0.3997	0.6808
CNN-LSTM (Focal Loss)	**0.5581**	**0.2702**	**0.7553**	**0.3980**	**0.7001**

The Focal Loss CNN-LSTM caught 318 out of 421 buggy modules in the test set (Recall: 75.5%), compared to 190 caught by the best baseline (RF + SMOTE, Recall: 45.1%) -- representing 128 additional bugs detected, a 67.4% improvement in Recall.

Figure 3: All models performance comparison



7. Next Steps

* **Notebook 05 -- BERT Model**

  Implement a BERT-based text classifier for bug report classification using the Bugzilla / GitBugs dataset.

* **Notebook 06 -- Enhanced CNN-LSTM**

  Fine-tune the Focal Loss model with additional epochs and class-weighted training to improve F1-Score while maintaining high Recall.

* **10-Fold Cross-Validation**

  Apply stratified k-fold CV across all models for statistical reliability.

* **Cross-Project Validation**

  Train on JM1, evaluate on KC1 to assess model generalisability.

* **Report Writing**

  Begin drafting Methodology and Results chapters.

Student:	S.Shanthosh (20APSE4882)
Supervisor:	Mrs. WMLS Abeythunga
Date:	27th May 2026